

1. តើ FastAPI គឺជាអ្វី?

FastAPI គឺជា Web Framework ទំនើបរបស់ Python ដែលត្រូវបានប្រើដើម្បីបង្កើត APIs បានយ៉ាងរហ័ស និងងាយស្រួល។ វាត្រូវបានបង្កើតឡើងដោយផ្អែកលើ៖

- Python 3.7+
- Type hints (ការកំណត់ប្រភេទ ទិន្នន័យ)
- Pydantic (សម្រាប់ការផ្ទៀងផ្ទាត់ទិន្នន័យ - Data validation)
- Starlette (Web framework មូលដ្ឋាន)



1.1 មូលហេតុដែល FastAPI មានប្រជាប្រិយភាព

FastAPI ត្រូវបានគេស្គាល់ដោយសារ៖

- ⚡ លឿនលឿនខ្លាំង: មានសមត្ថភាពខ្ពស់ (Performance) មិនចាញ់ Node.js ឬ Go ឡើយ។
- 📖 បង្កើតឯកសារ API ដោយស្វ័យប្រវត្តិ: បង្កើត Swagger UI (Interactive documentation) ឱ្យភ្លាមៗ។
- 🧠 ងាយស្រួលរៀន: កូដមានភាពសាមញ្ញ និងកាត់បន្ថយការសរសេរដដែលៗ។
- 🔒 មានប្រព័ន្ធផ្ទៀងផ្ទាត់ស្រាប់: ជួយពិនិត្យកំហុសទិន្នន័យបានល្អបំផុត។
- 🚀 ត្រៀមជាស្រេចសម្រាប់ដំណើរការ (Production-ready): ខ្លាំង និងមានស្ថេរភាព។

1.2 អ្វីដែលអ្នកអាចបង្កើតបានជាមួយ FastAPI

អ្នកអាចបង្កើតនូវ៖

- REST APIs
- ប្រព័ន្ធចុះឈ្មោះ និងចូលប្រើប្រាស់ (Login systems)
- OTP APIs (សម្រាប់ការផ្ទៀងផ្ទាត់លេខកូដតាមទូរស័ព្ទ)
- ផ្នែកខាងក្រោយនៃអាជីវកម្មអនឡាញ (E-commerce backend)
- ផ្នែកខាងក្រោយនៃកម្មវិធីទូរស័ព្ទ (Mobile app backend)
- Machine learning APIs (សម្រាប់ភ្ជាប់ជាមួយបញ្ហាសិប្បនិម្មិត)
- ប្រព័ន្ធទូទាត់ប្រាក់ (Payment APIs)

2. ការប្រៀបធៀប FastAPI vs Flask vs Django

លក្ខណៈពិសេស	FastAPI	Flask	Django
ល្បឿន (Speed)	⚡ លឿនខ្លាំង	មធ្យម	មធ្យម
កម្រិតនៃការរៀន	ងាយស្រួល	ងាយស្រួល	ពិបាក
ការផ្ទៀងផ្ទាត់ទិន្នន័យ	✅ មានស្រាប់	❌ មិនមាន	❌ មិនមាន
ឯកសារ API (Docs)	✅ ស្វ័យប្រវត្តិ	❌ សរសេរដោយដៃ	❌ សរសេរដោយដៃ
Async Support	✅ មានពេញលេញ	មានកម្រិត	មានកម្រិត
សាកសមបំផុតសម្រាប់	APIs	កម្មវិធីតូចៗ	គេហទំព័រពេញលេញ

2.1 🧠 តើគួរប្រើមួយណាជាម្រើសណា?

ប្រើប្រាស់ FastAPI ប្រសិនបើ៖

- អ្នកចង់បង្កើត APIs ។
- អ្នកត្រូវការ ល្បឿន និងសមត្ថភាពខ្ពស់ (High performance) ។
- អ្នកចង់បាន ឯកសារ API (Documentation) ដោយស្វ័យប្រវត្តិ។



2.1 🧠 តើគួរប្រើមួយណាជម្រើសណា?

ប្រើប្រាស់ Flask ប្រសិនបើ៖

- អ្នកចង់បង្កើត កម្មវិធីសាមញ្ញៗ។
- គម្រោងមាន ខ្នាតតូច។
- អ្នកចង់មានសេរីភាពក្នុងការរៀបចំ Structure ដោយខ្លួនឯង។



2.1 🧠 តើគួរប្រើមួយណាជាម្រឹមណា?

ប្រើប្រាស់ Django ប្រសិនបើ៖

- អ្នកចង់បង្កើត គេហទំព័រពេញលេញ (Full-stack websites)។
- ត្រូវការ ផ្ទាំងគ្រប់គ្រង (Admin panel) ដែលមានស្រាប់។
- ត្រូវការមុខងារច្រើនដូចជា Authentication និង Database ORM ដែលរៀបចំរួចជាស្រេច។

The Django logo, featuring the word "django" in a bold, dark green, lowercase sans-serif font. The letter 'j' has a distinctive hook that extends downwards and to the left.

3. ការដំឡើង FastAPI

- មុនពេលដំឡើង FastAPI សូមប្រាកដថាអ្នកបានដំឡើង Python រួចរាល់នៅក្នុងម៉ាស៊ីនរបស់អ្នក។

ត្រួតពិនិត្យកំណែ Python:

```
Bash
```

```
python --version
```

ឬ

```
Bash
```

```
python3 --version
```

3. ការដំឡើង FastAPI

- បង្កើត Virtual Environment (ត្រូវបានណែនាំឱ្យប្រើ) ការបង្កើតបរិស្ថានដាច់ដោយឡែកជួយកុំឱ្យជាន់គ្នាជាមួយគម្រោងផ្សេងទៀត៖

Bash



```
python -m venv venv
```

3. ការដំឡើង FastAPI

- របៀប Activate (បើកដំណើរការ):

- **Windows:**

Bash



```
venv\Scripts\activate
```

- **Mac / Linux:**

Bash



```
source venv/bin/activate
```

3. ការដំឡើង FastAPI

- ដំឡើង FastAPI

ប្រើប្រាស់ pip ដើម្បីដំឡើង៖

Bash



```
pip install fastapi
```

ត្រួតពិនិត្យការដំឡើង៖

Bash



```
pip show fastapi
```

4. ដំឡើង Uvicorn (Server)

- Uvicorn គឺជា ASGI server ដែលប្រើសម្រាប់ដំណើរការកម្មវិធី FastAPI របស់អ្នក។

របៀបដំឡើង:

Bash



```
pip install uvicorn
```

ការណែនាំបន្ថែម (Recommended):

ដើម្បីទទួលបានមុខងារពេញលេញ និងល្បឿនលឿន សូមដំឡើងកំណែ Standard:

Bash



```
pip install "uvicorn[standard]"
```

7. ប្រតិបត្តិការតាមផ្លូវ (Path Operations)

- នៅក្នុងពិភព API យើងប្រើ HTTP Methods ផ្សេងៗគ្នាដើម្បីប្រាប់ Server ថាអ្នកចង់ធ្វើអ្វី។ FastAPI គាំទ្រវិធីសាស្ត្រទាំងនោះយ៉ាងពេញលេញ៖

វិធីសាស្ត្រ (Method)	ការប្រើប្រាស់ (Use)	ការពន្យល់
GET	អានទិន្នន័យ (Read)	ប្រើសម្រាប់ទាញយកព័ត៌មានពី Server
POST	បង្កើតទិន្នន័យ (Create)	ប្រើសម្រាប់ធ្វើទិន្នន័យថ្មីទៅរក្សាទុក
PUT	ធ្វើបច្ចុប្បន្នភាព (Update)	ប្រើសម្រាប់កែប្រែទិន្នន័យដែលមានស្រាប់
DELETE	លុបទិន្នន័យ (Remove)	ប្រើសម្រាប់លុបទិន្នន័យចេញ

8. ប៉ារ៉ាម៉ែត្រតាមផ្លូវ (Path Parameters)

- Path Parameters ត្រូវបានប្រើដើម្បីបញ្ជូនតម្លៃទៅឱ្យ API តាមរយៈខ្សែផ្លូវនៃ URL ផ្ទាល់តែម្តង។

Python

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/users/{user_id}")
def get_user(user_id: int):
    return {"user_id": user_id}
```

- {user_id}: ជាកន្លែងដែលយើងកំណត់ថាជា Parameter។
- user_id: int: FastAPI ប្រើប្រាស់ Python Type Hints ដើម្បីកំណត់ថា តម្លៃដែលបញ្ជូនមកត្រូវតែជា ចំនួនគត់ (Integer)។ ប្រសិនបើអ្នកប្រើប្រាស់បញ្ជូនអក្សរមក វានឹងបង្ហាញកំហុស (Error) ដោយស្វ័យប្រវត្តិ។
- ឧទាហរណ៍ URL: `http://127.0.0.1:8000/users/10`

JSON

```
{
  "user_id": 10
}
```

8. ប៉ារ៉ាម៉ែត្រតាមផ្លូវ (Path Parameters)

- ការប្រើប្រាស់ Path Parameters ច្រើន (Multiple Path Parameters)

Python

```
@app.get("/users/{user_id}/posts/{post_id}")
def get_post(user_id: int, post_id: int):
    return {
        "user_id": user_id,
        "post_id": post_id
    }
```

- ឧទាហរណ៍ URL: <http://127.0.0.1:8000/users/5/posts/101>

JSON

```
{
    "user_id": 5,
    "post_id": 101
}
```

8. ប៉ារ៉ាម៉ែត្រតាមផ្លូវ (Path Parameters)

- ហេតុអ្វីបានជាវាអស្ចារ្យ?
 - Data Validation: ប្រសិនបើអ្នកព្យាយាមចូលទៅកាន់ `/users/abc` នោះ FastAPI នឹងប្រាប់ថា "value is not a valid integer" ភ្លាមៗ ព្រោះយើងបានកំណត់ប្រភេទវាជា `: int`។
 - Auto-Documentation: នៅក្នុង `/docs` (Swagger UI) វានឹងបង្កើតប្រអប់សម្រាប់ឱ្យអ្នកបញ្ចូល `user_id` និង `post_id` ដើម្បីតេស្តដោយស្វ័យប្រវត្តិ។
 - Clean URLs: វាធ្វើឱ្យ URL របស់អ្នកមានរបៀបរៀបរយ និងងាយស្រួលយល់ (ឧទាហរណ៍៖ `/products/5` ជាជាងប្រើ Query ដែលមានសញ្ញាសួរច្រើន)។

9. ប៉ារ៉ាម៉ែត្រសំណួរ (Query Parameters)

- Query Parameters គឺជាសំណុំនៃ Key-Value ដែលបង្ហាញនៅខាងក្រោយសញ្ញាសួរ ? នៅក្នុង URL។ ប្រសិនបើមានប៉ារ៉ាម៉ែត្រច្រើន គេប្រើសញ្ញា & ដើម្បីភ្ជាប់គ្នា។

Python

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/items/")
def get_items(limit: int = 10):
    return {"limit": limit}
```

- limit: int = 10: នេះមានន័យថា ប្រសិនបើអ្នកប្រើប្រាស់មិនបានបញ្ជូនតម្លៃ limit មកទេ វានឹងយកតម្លៃ 10 ជាលំនាំដើម។
- ឧទាហរណ៍ URL: <http://127.0.0.1:8000/items/?limit=5>

JSON

```
{
  "limit": 5
}
```

9.1 ការប្រើប្រាស់ Query Parameters ច្រើន

- អ្នកអាចដាក់ប៉ារ៉ាម៉ែត្រច្រើនដើម្បីធ្វើការ Filter ទិន្នន័យឱ្យកាន់តែលម្អិត៖

Python

```
@app.get("/products/")
def get_products(limit: int = 10, skip: int = 0):
    return {
        "limit": limit,
        "skip": skip
    }
```

- ឧទាហរណ៍ URL: <http://127.0.0.1:8000/products/?limit=5&skip=2>

JSON

```
{
  "limit": 5,
  "skip": 2
}
```

9.2. 🧐 ភាពខុសគ្នារវាង Path និង Query Parameters

- អ្នកអាចដាក់ប៉ារ៉ាម៉ែត្រច្រើនដើម្បីធ្វើការ Filter ទិន្នន័យឱ្យកាន់តែលម្អិត៖

លក្ខណៈ	Path Parameters	Query Parameters
ទម្រង់ URL	/users/10	/users/?id=10
សញ្ញាសម្គាល់	ប្រើ {} ក្នុងកូដ	ប្រើ ? ក្នុង URL
តម្រូវការ	ចាំបាច់ (Required)	ជាទូទៅគឺស្រេចចិត្ត (Optional)
ការប្រើប្រាស់	កំណត់អត្តសញ្ញាណ (Identify)	ចម្រោះ/តម្រៀប (Filter/Sort/Page)

- ឧទាហរណ៍ URL: `http://127.0.0.1:8000/products/?limit=5&skip=2`

JSON

```
{  
  "limit": 5,  
  "skip": 2  
}
```

9.3. ចំណុចគួរដឹង៖

- FastAPI ធ្លាត់វែណាស់! ប្រសិនបើអ្នកប្រកាស Parameter ក្នុងអនុគមន៍ដែល មិនមាន នៅក្នុង URL Path (@app.get("/...")) នោះ FastAPI នឹងចាត់ទុកវាជា Query Parameter ដោយស្វ័យប្រវត្តិ។

10. 🎯 លំហាត់អនុវត្តខ្លីៗ (Mini Practice Tasks)

- សាកល្បងសរសេរបន្ថែមក្នុង main.py ដើម្បីពង្រឹងសមត្ថភាព៖
- កិច្ចការទី ១: បង្កើត route /hello ដែលបញ្ជូនលទ្ធផល {"message": "Hello World"}។
- កិច្ចការទី ២: បង្កើត /students/{id} ដែលទទួលតម្លៃ ID ជាលេខ រួចបង្ហាញវិញតាមរយៈ {"student_id": id}។
- កិច្ចការទី ៣: បង្កើត /search/ ដែលទទួល Query parameter ឈ្មោះ keyword (ឧទាហរណ៍៖ /search/?keyword=python)។



11. Request Body (ការផ្ញើទិន្នន័យតាមរយៈ POST)

Request Body ត្រូវបានប្រើដើម្បីផ្ញើទិន្នន័យដែលមានទម្រង់ស្មុគស្មាញ (ដូចជាព័ត៌មានចុះឈ្មោះ ឬព័ត៌មានទំនិញ) ដែលមិនអាចផ្ញើតាមរយៈ URL បានឡើយ។

វាត្រូវបានប្រើប្រាស់ជាទូទៅបំផុតក្នុង៖

- ការចុះឈ្មោះអ្នកប្រើប្រាស់ (User registration)
- ការចូលប្រើប្រាស់ (Login)
- ការបង្កើតផលិតផលថ្មីៗ
- ការរក្សាទុកទិន្នន័យផ្សេងៗ

11.1 ជំហានទី ១ — ស្វែងយល់ពី Pydantic

FastAPI ប្រើប្រាស់ Pydantic ដើម្បីកំណត់ "រូបរាង" ឬ "គ្រោង" នៃទិន្នន័យដែលយើងចង់បាន។
វាជួយពិនិត្យថា តើទិន្នន័យដែលគេផ្ញើមកនោះត្រឹមត្រូវតាមលក្ខខណ្ឌដែលយើងចង់បានដែរឬទេ?

Python

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

# បង្កើត Model ដើម្បីកំណត់ទម្រង់ទិន្នន័យ
class User(BaseModel):
    name: str # ត្រូវតែជាអក្សរ
    age: int # ត្រូវតែជាលេខ
    email: str # ត្រូវតែជាអក្សរ

@app.post("/users/")
def create_user(user: User):
    # យើងអាចទាញយកទិន្នន័យបានតាមរយៈ: user.name, user.age ...
    return {
        "status": "ជោគជ័យ",
        "data_received": {
            "name": user.name,
            "age": user.age,
            "email": user.email
        }
    }
```

 របៀបតេស្តក្នុង Swagger UI

1. ចូលទៅកាន់: <http://127.0.0.1:8000/docs>
2. ចុចលើពាក្យ POST /users/ រួចចុច Try it out។
3. នៅក្នុងប្រអប់ Request body សូមសាកល្បងដាក់ទិន្នន័យនេះ៖

JSON

```
{
  "name": "Hak",
  "age": 20,
  "email": "hak@gmail.com"
}
```

4. ចុច Execute រួចមើលលទ្ធផលខាងក្រោម។

11.2 ហេតុអ្វីបានជាវាសំខាន់?

- Automatic Validation: ប្រសិនបើអ្នកផ្ញើ "age": "ម្ភៃ" (ជាអក្សរ) មកវិញ FastAPI នឹងប្រាប់ ក្លាមថា Error ព្រោះវាត្រូវការជាលេខ (int)។
- Auto-Completion: ពេលអ្នកសរសេរកូដក្នុង VS Code អ្នកគ្រាន់តែវាយ user. វានឹងបង្ហាញ ឈ្មោះ name, age, email ឱ្យអ្នកជ្រើសរើសភ្លាមៗ (Intellisense)។
- Security: វាជួយការពារការធ្វើទិន្នន័យដែលមិនស្គាល់អត្តសញ្ញាណចូលមកក្នុងប្រព័ន្ធរបស់ អ្នក។

14. ការប្រើប្រាស់វាលទិន្នន័យច្រើនក្នុង Request Body

នៅពេលយើងបង្កើត Model សម្រាប់ផលិតផល (Product) យើងត្រូវការកំណត់ប្រភេទដេតាឱ្យបានច្បាស់លាស់ ដើម្បីឱ្យ FastAPI ជួយត្រួតពិនិត្យ (Validation) ឱ្យយើង។

Python

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

# បង្កើត Model សម្រាប់ផលិតផល
class Product(BaseModel):
    name: str      # ឈ្មោះផលិតផល (អក្សរ)
    price: float    # តម្លៃ (លេខមានក្បៀស ឧទាហរណ៍: 12.50)
    stock: int      # ចំនួនក្នុងស្តុក (ចំនួនគត់)

@app.post("/products/")
def create_product(product: Product):
    # នៅទីនេះ product គឺជា Object ដែលមាន data ទាំងអស់
    # អ្នកអាចបញ្ជូន (Return) វាមកវិញទាំងមូលតែម្តងក៏បាន
    return product
```

🔍 ការបកស្រាយលម្អិត៖

1. price: float: ការប្រើ float អនុញ្ញាតឱ្យអ្នកប្រើប្រាស់ផ្ញើតម្លៃដូចជា 9.99 ឬ 15.0។ ប្រសិនបើគេផ្ញើជាលេខគត់ 10 វានឹងបំប្លែងទៅជា 10.0 ដោយស្វ័យប្រវត្តិ។
2. stock: int: ចំនួនទំនិញមិនអាចមានក្បៀសបានទេ ដូច្នេះយើងប្រើ int។ ប្រសិនបើគេផ្ញើ 5.5 មក FastAPI នឹងផ្ញើ Error ទៅវិញថាជាទិន្នន័យមិនត្រឹមត្រូវ។
3. return product: FastAPI ឆ្លាតវៃណាស់! វាបំប្លែង Pydantic Object ទៅជា JSON format ឱ្យយើងដោយស្វ័យប្រវត្តិដើម្បីផ្ញើទៅ Client វិញ។

15 ការប្រើប្រាស់វាលទិន្នន័យស្របចិត្ត (Optional Fields)

ដើម្បីប្រើ Optional អ្នកត្រូវនាំចូល (Import) វាពីបណ្ណាល័យ typing របស់ Python។

```
Python

from fastapi import FastAPI
from pydantic import BaseModel
from typing import Optional

app = FastAPI()

class User(BaseModel):
    name: str
    age: int
    # email អាចជា string ក៏បាន ឬមិនបញ្ជូនមក (None) ក៏បាន
    email: Optional[str] = None

@app.post("/register")
def register_user(user: User):
    if user.email:
        return {"message": f"ចុះឈ្មោះ {user.name} ជាមួយអ៊ីមែល {user.email} រួចរាល់"}
    return {"message": f"ចុះឈ្មោះ {user.name} រួចរាល់ (មិនមានអ៊ីមែល)"}

```

🔍 ការបកស្រាយ៖

1. Optional[str] = None:

- វាប្រាប់ FastAPI ថា email មិនមែនជាទិន្នន័យបង្ខំ (Required) នោះទេ។
- ប្រសិនបើអ្នកប្រើប្រាស់មិនបានផ្ញើ email មកក្នុង JSON Body ទេ នោះតម្លៃរបស់វានឹងក្លាយជា None ដោយស្វ័យប្រវត្តិ។

2. Validation:

- name និង age នៅតែជាទិន្នន័យចាំបាច់។ បើមិនផ្ញើមក វានឹង Error។
- email បើផ្ញើមក ត្រូវតែជាប្រភេទ str (អក្សរ)។

16. ការប្រើប្រាស់ Path Parameters និង Request Body រួមគ្នា

ជាទូទៅ នៅពេលអ្នកចង់កែប្រែព័ត៌មានអ្វីមួយ អ្នកត្រូវប្រាប់ Server នូវរឿងពីរយ៉ាង៖

1. តើអ្នកចង់កែប្រែទិន្នន័យរបស់នរណា? (បញ្ជូនតាមរយៈ user_id ក្នុង URL)
2. តើទិន្នន័យថ្មីនោះមានអ្វីខ្លះ? (បញ្ជូនតាមរយៈ User model ក្នុង Request Body)

```
Python

from fastapi import FastAPI
from pydantic import BaseModel
from typing import Optional

app = FastAPI()

class User(BaseModel):
    name: str
    age: int
    email: Optional[str] = None

# ការប្រើប្រាស់: user_id បានមកពី URL, ចំណែក user បានមកពី JSON Body
@app.put("/users/{user_id}")
def update_user(user_id: int, user: User):
    return {
        "message": f"បានធ្វើបច្ចុប្បន្នភាពអ្នកប្រើប្រាស់លេខ {user_id} រួចរាល់",
        "user_id": user_id,
        "updated_data": user
    }
```

🔍 ការបកស្រាយ៖

- user_id: int: FastAPI នឹងស្វែងរកតម្លៃនេះនៅក្នុង URL Path {user_id}។
- user: User: ដោយសារ User គឺជា Pydantic Model នោះ FastAPI នឹងដឹងដោយស្វ័យប្រវត្តិថាត្រូវអានទិន្នន័យចេញពី Request Body (JSON)។

17. ការប្រើប្រាស់ Query Parameters និង Request Body រួមគ្នា

នៅក្នុងឧទាហរណ៍នេះ យើងនឹងធ្វើទិន្នន័យផលិតផលតាមរយៈ JSON Body ប៉ុន្តែយើងក៏ចង់បញ្ចូលតម្លៃ "បញ្ចុះតម្លៃ" (discount) តាមរយៈ URL ផងដែរ។

```
Python

from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Product(BaseModel):
    name: str
    price: float
    stock: int

@app.post("/items/")
def create_item(
    item: Product,          # បានមកពី Request Body (JSON)
    discount: float = 0     # បានមកពី Query Parameter (?discount=5)
):
    # គណនាតម្លៃក្រោយបញ្ចុះតម្លៃ
    final_price = item.price - discount

    return {
        "item_name": item.name,
        "original_price": item.price,
        "discount_applied": discount,
        "final_price": final_price
    }
```

🔍 ការបកស្រាយ៖

- item: Product: ដោយសារវាជា Pydantic Model នៅ៖ FastAPI ដឹងថាវាត្រូវតែទាញចេញពី Request Body។
- discount: float = 0: ដោយសារវាជាប្រភេទលេខ (float) សាមញ្ញ ហើយវាមិនមានប្រកាសនៅក្នុង URL Path ទេ នៅ៖ FastAPI ចាត់ទុកវាជា Query Parameter ដោយស្វ័យប្រវត្តិ។

18. ការដោះស្រាយទិន្នន័យទម្រង់បែបបទ (Form Data Handling)

ជាធម្មតា នៅពេលអ្នកបង្កើតវេបសាយដែលមានប្រអប់បញ្ចូលអក្សរ (Input Fields)

ហើយចុចប៊ូតុង Submit នោះទិន្នន័យនឹងត្រូវបានផ្ញើមកក្នុងទម្រង់ "Form Data"។

វាត្រូវបានប្រើប្រាស់សម្រាប់៖

- ទម្រង់ HTML ស្តង់ដារ (Standard HTML forms)
- ប្រព័ន្ធចូលប្រើប្រាស់ (Login forms)
- ការផ្ញើទិន្នន័យដែលមានលាយឡំជាមួយឯកសារ (Files)

18.1 🛠️ ជំហានទី ១ — ដំឡើងកញ្ចប់ចាំបាច់

ដើម្បីឱ្យ FastAPI អាចអាន Form Data បាន អ្នកត្រូវដំឡើងបណ្ណាល័យ python-multipart ជាមុនសិន៖

Bash

```
pip install python-multipart
```

Python

```
from fastapi import FastAPI, Form

app = FastAPI()

@app.post("/login/")
def login(
    username: str = Form(...), # សញ្ញា (...) មានន័យថាជាទិន្នន័យបង្ខំ (Required)
    password: str = Form(...)
):
    # ក្នុងករណីពិត អ្នកនឹងយក username/password ទៅធ្វើការផ្ទៀងផ្ទាត់ក្នុង Database
    return {
        "username": username,
        "password": password
    }
```

🔍 ការបកស្រាយ៖

- Form(...): ប្រាប់ FastAPI ថាទិន្នន័យនេះមិនមែនបានមកពី JSON Body ឬ Query ទេ ប៉ុន្តែបានមកពី Body Form Fields។
- សុវត្ថិភាព: ការប្រើ POST ជាមួយ Form សម្រាប់ Login គឺមានសុវត្ថិភាពជាងការប្រើ Query Parameters ព្រោះវាមិនបង្ហាញ Password នៅលើ URL ឡើយ។

19. ការបង្ហាត់ឯកសារ (File Upload)

នៅក្នុង FastAPI យើងមានជម្រើសពីរគឺ File (សម្រាប់ទិន្នន័យតូចៗជា bytes) និង UploadFile (ដែលប្រសើរជាងសម្រាប់ការបង្ហាត់ឯកសារទូទៅ)។

ហេតុអ្វីបានជាគេនិយមប្រើ UploadFile?

- វាមិនរក្សាទុកឯកសារទាំងស្រុងក្នុង Memory ទេ (ជួយឱ្យ Server មិនគាំងពេល File ធំ)។
- អ្នកអាចទាញយកព័ត៌មានបន្ថែមដូចជា ឈ្មោះឯកសារ (filename) និងប្រភេទឯកសារ (content_type) បានយ៉ាងងាយ។

19. ការបង្ហាត់ឯកសារ (File Upload)

 ឧទាហរណ៍ — ការបង្ហាត់ឯកសារមួយ (Single File)

Python

```
from fastapi import FastAPI, UploadFile, File

app = FastAPI()

@app.post("/upload/")
async def upload_file(file: UploadFile = File(...)):
    # អ្នកអាចអានទិន្នន័យក្នុង file បានតាមរយៈ: file.read()
    return {
        "filename": file.filename,
        "content_type": file.content_type
    }
```

19. ការបង្ហាត់ឯកសារ (File Upload)

🔗 ការបង្ហាត់ឯកសារច្រើនក្នុងពេលតែមួយ (Multiple Files)

Python

```
from typing import List
from fastapi import FastAPI, UploadFile, File

app = FastAPI()

@app.post("/upload-multiple/")
async def upload_multiple(files: List[UploadFile] = File(...)):
    # បង្កើតបញ្ជីឈ្មោះឯកសារដែលបានបង្ហាត់
    filenames = [file.filename for file in files]

    return {
        "count": len(files),
        "filenames": filenames
    }
```

🔍 ចំណុចសំខាន់ៗដែលត្រូវដឹង៖

- 1.async def: ការបង្ហាត់ឯកសារគឺជាកិច្ចការដែលត្រូវការពេល (I/O operation) ដូច្នេះការប្រើ async ជួយឱ្យ FastAPI ដំណើរការបានលឿន និងមិនរង់ចាំគ្នា។
- 2.File(...): ប្រាប់ FastAPI ថាទិន្នន័យនេះត្រូវតែជាឯកសារ (មិនមែនជា JSON ឬ Form ធម្មតា)។
- 3.Validation: អ្នកអាចឆែកមើលប្រភេទឯកសារបាន។ ឧទាហរណ៍៖ បើ file.content_type មិនមែនជា image/jpeg អ្នកអាចបដិសេធមិនឱ្យគេ Upload បាន។

20. ការរក្សាទុកឯកសារទៅក្នុងម៉ាស៊ីន (Save to Disk)

ដើម្បីរក្សាទុកឯកសារ យើងប្រើបណ្ណាល័យ `shutil` ដែលមានស្រាប់ក្នុង Python សម្រាប់ចម្លងទិន្នន័យពីឯកសារដែលបានបង្ហាត់ (Temporary file) ទៅកាន់ទីតាំងថ្មីដែលយើងចង់បាន។

Python

```
import shutil
from fastapi import FastAPI, UploadFile, File

app = FastAPI()

@app.post("/save-file/")
def save_file(file: UploadFile = File(...)):
    # ១. បង្កើត ឬបើកឯកសារថ្មីក្នុងរបៀប "wb" (Write Binary)
    # យើងប្រើ file.filename ជាឈ្មោះឯកសារដែលរក្សាទុក
    with open(file.filename, "wb") as buffer:
        # ២. ចម្លងទិន្នន័យពី file.file (Uploaded object) ទៅកាន់ buffer (ឯកសារថ្មី)
        shutil.copyfileobj(file.file, buffer)

    return {
        "message": "រក្សាទុកឯកសារជោគជ័យ",
        "filename": file.filename
    }
```

🔍 ការបកស្រាយបច្ចេកទេស៖

1. `import shutil`: ជាបណ្ណាល័យសម្រាប់ប្រតិបត្តិការលើឯកសារ (File operations) ដូចជាការចម្លង ឬផ្លាស់ទី។
2. `with open(...) as buffer`: នេះគឺជាការបើកឯកសារក្នុង Python។ យើងប្រើ `wb` (Write Binary) ព្រោះឯកសារគ្រប់ប្រភេទ (រូបភាព, PDF, វីដេអូ) ត្រូវបានរក្សាទុកជាទម្រង់ Binary។
3. `shutil.copyfileobj(file.file, buffer)`: នេះគឺជាបេះដូងនៃកូដនេះ។ វាធ្វើការចម្លងទិន្នន័យជាកញ្ចប់ៗ (Streams) ពីកន្លែងផ្ញើមកកាន់ទីតាំងផ្ទុក។ វាមានប្រសិទ្ធភាពខ្ពស់ ទោះបីជាឯកសារមានទំហំធំក៏ដោយ។

21. ការប្រើប្រាស់ Parameters ប្រើប្រភេទទូទៅ

នៅក្នុងឧទាហរណ៍នេះ យើងនឹងបង្កើត Endpoint មួយដែលទទួលយក៖

1. Path Parameter: {product_id} (ដើម្បីកំណត់លេខសម្គាល់)
2. Request Body: Product model (សម្រាប់ឈ្មោះ និងតម្លៃ)
3. File: UploadFile (សម្រាប់រូបភាពផលិតផល)

21. ការប្រើប្រាស់ Parameters ប្រើប្រភេទ ទូរមគ្គ

Python

```
from fastapi import FastAPI, UploadFile, File, Query
from pydantic import BaseModel

app = FastAPI()

class Product(BaseModel):
    name: str
    price: float

@app.post("/products/{product_id}")
async def create_product(
    product_id: int,                # Path Parameter
    product: Product,              # Request Body (JSON)
    file: UploadFile = File(...),  # File Upload
    category: str = Query("General"), # Query Parameter (ស្រេចចិត្ត)
):
    return {
        "message": "ទិន្នន័យត្រូវបានទទួល",
        "product_id": product_id,
        "product_details": product,
        "category": category,
        "filename": file.filename,
        "content_type": file.content_type
    }
```

🔍 ការបកស្រាយអំពីការដោះស្រាយទិន្នន័យ (Data Handling):

FastAPI មានភាពឆ្លាតវៃក្នុងការបែងចែកទិន្នន័យដែលផ្ញើមក៖

- JSON Body & Files: នៅពេលអ្នកប្រើទាំង BaseModel និង File រួមគ្នា FastAPI នឹងរៀបចំទម្រង់ផ្ញើជា multipart/form-data ដោយស្វ័យប្រវត្តិ។
- Validation: ប្រសិនបើ product_id មិនមែនជាលេខ ឬ file មិនត្រូវបានជ្រើសរើស នោះ API នឹងបដិសេធ (Reject) សំណើនោះភ្លាមៗ ជាមួយនឹងសារកំហុសដ៏ច្បាស់លាស់។

22. ការផ្ទៀងផ្ទាត់សំណើ (Request Validation)

FastAPI ប្រើប្រាស់ Type Hints របស់ Python ដើម្បីធ្វើ Validation ដោយស្វ័យប្រវត្តិ។ នៅពេលមានទិន្នន័យផ្ញើមក វានឹងធ្វើការពិនិត្យទៅតាមជំហានដូចខាងក្រោម៖

Python

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class User(BaseModel):
    name: str
    age: int  # បង្ខំឱ្យជាលេខគត់
```

• Input (JSON):

JSON

```
{
  "name": "Sok",
  "age": "២៥"
}
```

• Response (Error 422):

FastAPI នឹងបង្ហាត់សំណើនោះភ្លាមៗ ដោយមិនឱ្យកូដខាងក្នុងដំណើរការឡើយ៖

JSON

```
{
  "detail": [
    {
      "loc": ["body", "age"],
      "msg": "value is not a valid integer",
      "type": "type_error.integer"
    }
  ]
}
```

ចំណុចពិសេសនៃ Validation ក្នុង FastAPI

1. Data Conversion (Coercion): វាមិនមែនតែប្រភេទដេតាប្រភេទនោះទេ តែវាជួយបំប្លែងឱ្យទៀតផង។ ប្រសិនបើអ្នកផ្ញើ "age": "20" (ជាអក្សរតែមានលក្ខណៈជាលេខ) វានឹងបំប្លែងទៅជាលេខ 20 (integer) ឱ្យអ្នកដោយស្វ័យប្រវត្តិ។
2. Clear Error Messages: វានឹងប្រាប់អ្នកប្រើប្រាស់ចំៗថា តើ Error នៅត្រង់ណា (loc), មូលហេតុអ្វី (msg), និងប្រភេទអ្វី (type)។
3. Automatic Documentation: លក្ខខណ្ឌ Validation ទាំងអស់ដែលអ្នកសរសេរ នឹងបង្ហាញនៅក្នុង Swagger UI (/docs) ដោយស្វ័យប្រវត្តិ ដើម្បីឱ្យអ្នកប្រើប្រាស់ដឹងថាគួរផ្ញើអ្វីមកខ្លះ។

23. ការអានព័ត៌មានពី Headers

នៅក្នុង FastAPI យើងប្រើ Header ពី module fastapi ដើម្បីទាញយកតម្លៃទាំងនោះ។

Python

```
from fastapi import FastAPI, Header
from typing import Optional

app = FastAPI()

@app.get("/headers/")
def read_headers(
    # FastAPI នឹងបំប្លែងពី "User-Agent" ទៅជា "user_agent" ដោយស្វ័យប្រវត្តិ
    user_agent: Optional[str] = Header(None),
    accept_language: Optional[str] = Header(None),
    custom_id: Optional[str] = Header(None)
):
    return {
        "User-Agent": user_agent,
        "Language": accept_language,
        "Custom-ID": custom_id
    }
```

🔍 ចំណុចពិសេសដែលត្រូវដឹង

1. Automatic Conversion: ជាទូទៅ Headers ត្រូវបានសរសេរដោយប្រើសញ្ញាដក (ឧទាហរណ៍៖ User-Agent) ប៉ុន្តែក្នុង Python យើងមិនអាចប្រើសញ្ញាដកក្នុងឈ្មោះ Variable បានទេ។ FastAPI ឆ្លាតវៃណាស់ វានឹងបំប្លែងសញ្ញា _ (Underscore) ទៅជា - (Hyphen) ដោយស្វ័យប្រវត្តិដើម្បីទៅស្វែងរកក្នុង Header។
 - user_agent 🔍 ស្វែងរកក្នុង Header ឈ្មោះ User-Agent
2. Case Insensitive: វាមិនប្រកាន់អក្សរតូច ឬធំឡើយ (user-agent ឬ User-Agent គឺដូចគ្នា)។
3. Security & Auth: Headers ត្រូវបានប្រើប្រាស់ច្រើនបំផុតសម្រាប់ធ្វើ API Keys ឬ Bearer Tokens ដើម្បីផ្ទៀងផ្ទាត់សិទ្ធិចូលប្រើប្រាស់។

24. ការអានព័ត៌មានពី Cookies

នៅក្នុង FastAPI យើងប្រើ Cookie ពី module fastapi ដើម្បីទាញយកតម្លៃដែលផ្ញើមកពី

Browser។

Python

```
from fastapi import FastAPI, Cookie
from typing import Optional

app = FastAPI()

@app.get("/cookies/")
def read_cookie(
    # ត្រូវបានកំណត់ថា session_id
    session_id: Optional[str] = Cookie(None)
):
    if not session_id:
        return {"message": "អ្នកមិនមាន Cookie ទេ"}

    return {
        "session_id": session_id
    }
```

🔍 ចំណុចសំខាន់ៗអំពី Cookies

1. Security: Cookies ត្រូវបានប្រើសម្រាប់រក្សា session_id។ នៅពេល User ចូលប្រើប្រាស់ (Login) ជាមួយ Server នឹងផ្ញើ Cookie ទៅឱ្យ Browser រក្សាទុក ហើយរាល់ពេល User ចូលមើលទំព័រផ្សេងៗ Browser នឹងផ្ញើ Cookie នោះមកវិញដោយស្វ័យប្រវត្តិ។
2. Automatic Detection: ដូច Header ដែរ FastAPI នឹងស្វ័យប្រវត្តិរកឈ្មោះ Parameter (ក្នុងឧទាហរណ៍គឺ session_id) នៅក្នុងបញ្ជី Cookies ដែល Browser ផ្ញើមក។
3. Difference from Header: Header ជារឿយៗត្រូវបានកំណត់ដោយអ្នកសរសេរកូដ (Manual), ចំណែក Cookie ត្រូវបានគ្រប់គ្រងដោយ Browser (Automatic) បន្ទាប់ពីវាត្រូវបានកំណត់ម្តងរួចមក។

25. ដំណោះស្រាយសម្រាប់ Practice Tasks

កិច្ចការទី ១ (Login API):

- Route: /login (Method: POST)
- បច្ចេកទេស: ប្រើ Form(...) ដើម្បីទទួលទិន្នន័យ។ វាក្លែងធ្វើដូចជាទម្រង់ចុះឈ្មោះលើវេបសាយពិតៗ។ Password នឹងមិនបង្ហាញនៅលើ URL ឡើយ ដែលវាមានសុវត្ថិភាពជាងការប្រើ Query Parameters។

កិច្ចការទី ២ (Upload Profile):

- Route: /upload-profile (Method: POST)
- បច្ចេកទេស: ប្រើ UploadFile និង File(...)។ វាអនុញ្ញាតឱ្យអ្នកជ្រើសរើសរូបភាពពីក្នុងម៉ាស៊ីនរួចផ្ញើទៅកាន់ Server។ អ្នកអាចដឹងពីឈ្មោះឯកសារ និងប្រភេទ (ដូចជា image/png)។

កិច្ចការទី ៣ (Product API):

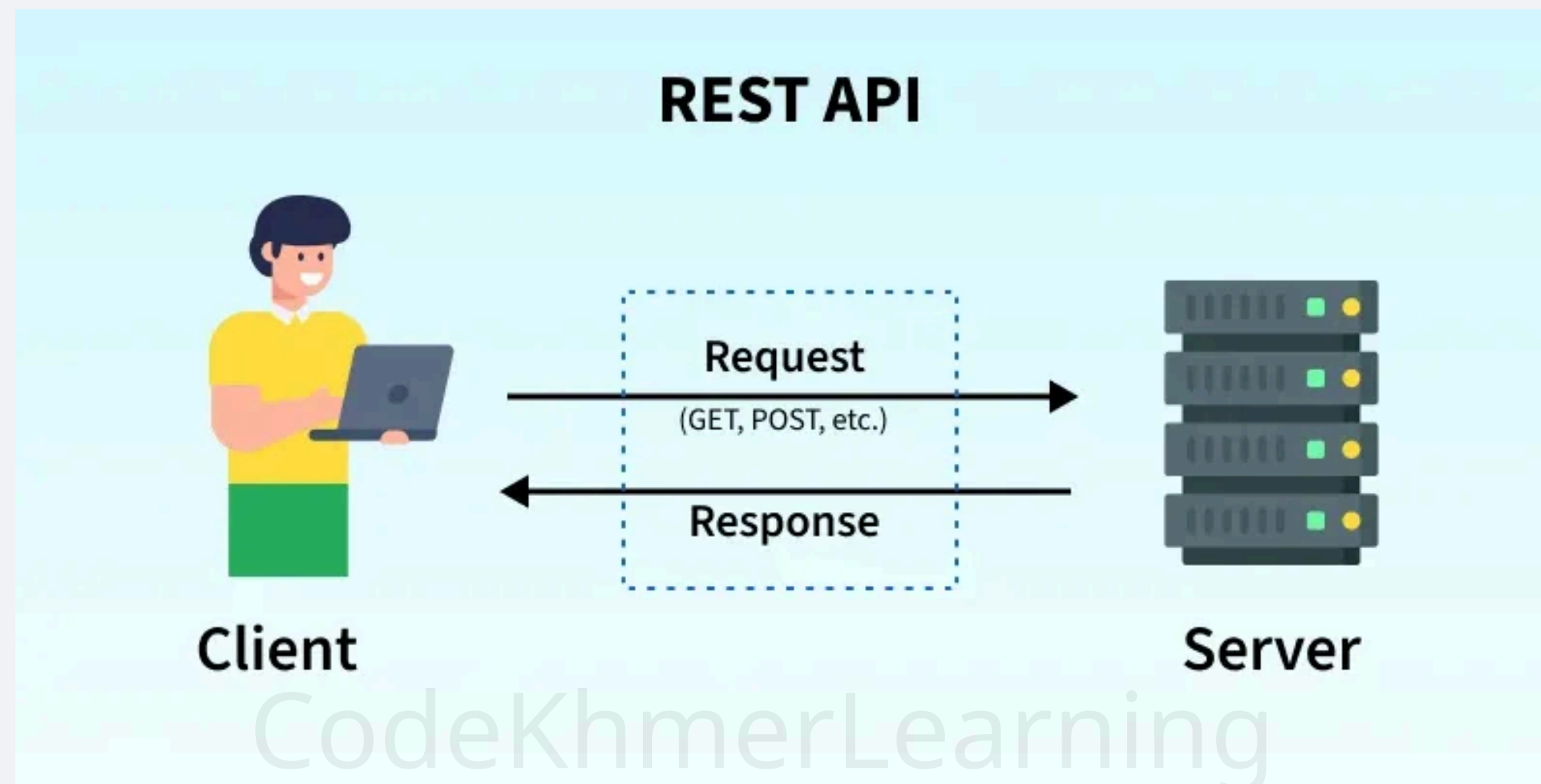
- Route: /products (Method: POST)
- បច្ចេកទេស: ប្រើ BaseModel របស់ Pydantic។ នេះជាវិធីដែលនិយមបំផុតសម្រាប់ API ទំនើបៗ ព្រោះវាអនុញ្ញាតឱ្យផ្ញើទិន្នន័យជាកញ្ចប់ JSON។ ខ្ញុំបានបន្ថែម status_code=201 ដើម្បីប្រាប់ថា "ទិន្នន័យត្រូវបានបង្កើតថ្មីដោយជោគជ័យ"។

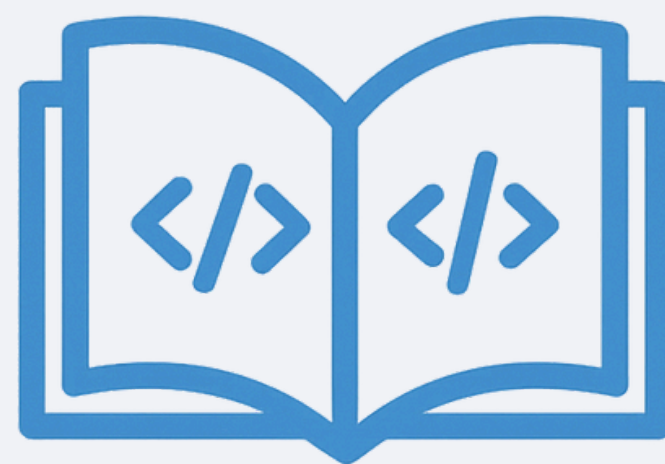
25. 🧪 ដំណោះស្រាយសម្រាប់ Practice Tasks

គម្រោងតូច៖ Simple User API

នៅក្នុងគម្រោងនេះ យើងមាន Endpoint ចំនួនពីរ៖

1. POST /users/: សម្រាប់ទទួលទិន្នន័យអ្នកប្រើប្រាស់ថ្មី រួចរក្សាទុកក្នុងបញ្ជី users។
2. GET /users/: សម្រាប់ទាញយកបញ្ជីអ្នកប្រើប្រាស់ទាំងអស់ដែលបានរក្សាទុកមកបង្ហាញវិញ។





CodeKhmer Learning

ក្បួនសិទ្ធផោយ កូដខ្មែរលេងនឹង

CodeKhmerLearning